Phytium飞腾

飞腾嵌入式 optee 的安全存储

（V1.2）

飞腾信息技术有限公司
www.phytium.com.cn
2023 年 05 月 12 日

版权所有 © 飞腾信息技术有限公司 2023。 保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区： https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台： https://service.phytium.com.cn/developer/

目录

1 安全存储 1
1.1 概述 1
1.2 软件调用框架 1
1.3 密钥派生 2
1.3.1 概述 2
1.3.2 SSK 3
1.3.3 TSK 3
1.3.4 FEK 4
1.4 Hashtree 4
2 RPMB 6
2.1 硬件属性 6
2.2 读写流程 6
2.2.1 概述 6
2.2.2 密钥 7
2.2.3 读流程 7
2.2.4 写流程 7
2.3 重放攻击 8
2.4 增强安全性 9
3 基于 xtest 的测试 10
3.1 前期配置 10
3.2 常见 log 和错误说明 10
3.2.1 测试 log 10
3.2.2 OTP 没有 key 的错误 13
3.2.3 RPMB key 验证失败的错误 13
3.2.4 RPMB_EMU 忘记关闭的错误 13
A 术语和缩略语 15
B 更新记录 16

1 安全存储

1.1 概述

本文档主要基于硬件环境为飞腾腾珑E2000Q参考板，TEE侧安全操作系统为OPTEE v3.12的平台，对安全存储，重点是对RPMB方向，进行了一些研究和测试。

1.2 软件调用框架

软件调用框架图请参考https://optee.readthedocs.io文档中的REE FS Secure Storage小节的第一张图。

OPTEE安全存储支持除eMMC的RPMB分区模式之外，还可以支持REE FS模式，本文档只重点介绍RPMB方式。

安全存储以RPMB方式为例，目标用户是TEE侧的TA程序。

1) GP Internal API Library

向TA程序提供符合GP标准的API接口，在GP标准中，这一块API定义为Trusted Storage（可信存储）API，实质是和OPTEE定义的Secure Storage（安全存储）是一样的，只是叫法不同，接口在libutee中。

2) TEE Trusted Storage Svc. Call

GP的API向OPTEE OS要求多个syscall的接口，OPTEE OS完成相关需求，OPTEE3.12提供的所有相关接口如下图所示：

```
SYSCALL_ENTRY(syscall_storage_obj_open),
SYSCALL_ENTRY(syscall_storage_obj_create),
SYSCALL_ENTRY(syscall_storage_obj_del),
SYSCALL_ENTRY(syscall_storage_obj_rename),
SYSCALL_ENTRY(syscall_storage_alloc_enum),
SYSCALL_ENTRY(syscall_storage_free_enum),
SYSCALL_ENTRY(syscall_storage_reset_enum),
SYSCALL_ENTRY(syscall_storage_start_enum),
SYSCALL_ENTRY(syscall_storage_next_enum),
SYSCALL_ENTRY(syscall_storage_obj_read),
SYSCALL_ENTRY(syscall_storage_obj_write),
SYSCALL_ENTRY(syscall_storage_obj_trunc),
SYSCALL_ENTRY(syscall_storage_obj_seek),
```

代码的具体实现是在tee_svc_storage.c中。

3) TEE File Operation Interface

OPTEE 将存储的实体看出一个 storage obj 进行操作，在内核中按照一种 tee obj 进行面向对象的管理，tee obj 总的定义在 tee_obj.h 中定义。作为文件操作的函数指针定义在 pobj（persistant object）中的 tee_file_operations 结构，这就是 OPTEE 统一的文件操作接口。

4) TEE File System

tee_file_operations 是 TEE 文件系统的核心结构，定义了 11 种基本文件和文件夹操作的函数指针。这里相当于完成了一个文件系统的上层框架，后续需要将实际的设备挂接到这个接口中，目前 OPTEE 支持两种实际设备，分别是 REE_FS 和 RPMB_FS，默认是 REE_FS，可以配置成 RPMB_FS，根据代码，两者是不可以同时有效，且必须有一种有效，即 TEE 文件系统必须有且只有一个实体挂接。因为没有 VFS，所以无法像 Linux 那样不同实体挂接在不同目录之下。

5) REE File Operation Interface

这里 REE 文件操作并不是单指 REE_FS，因为 RPMB_FS 也是要通过 REE 侧的文件操作来支持。REE_FS 的实际操作驱动在 tee_ree_fs.c 代码中，RPMB_FS 的实际操作驱动在 tee_rpmb_fs.c 代码中。无论是哪一种，都会归结到 RPC 服务中。RPC 服务是 OPTEE 和 REE 侧的 supplicant 后台程序交互的一种通讯方式，通过该服务，将文件的实际操作发送到 supplicant 中处理。

6) TEE Supplicant

tee-supplicant 是 OPTEE 运行在 Linux 应用层的后台程序，负责处理 OPTEE 发送过来的请求，其中 RPMB_FS 的相关请求是 rpmb.c 代码中进行处理的。在这里，实现了一个在内存空间中模拟的仿真 RPMB 环境，也支持通过标准 POSIX 接口访问 Linux 的 RPMB 设备。后续就是 Linux 相关设备驱动需要完成的事情。

1.3 密钥派生

1.3.1 概述

OPTEE 安全存储框架的核心就是将 TEE 侧的文件加密存储在 REE 侧的设备中。那么怎么确保安全，密钥的管理和派生（Key Manager）就是重中之重。

关于安全存储的密钥有三把，分别是：

1) SSK（Secure Storage Key）
2) TSK（Trusted Application Storage Key）
3) FEK（File Encryption Key）

这三把密钥都是通过一定的规则由上一级密钥派生出来的。

OPTEE 密钥派生的主函数是 huk_subkey_derive，在 huk_subkey.c 中实现。

Key manager 的核心代码在 tee_fs_key_manager.c 中实现。

1.3.2 SSK

OPTEE 在系统启动的时候会初始化 Key manager，这时候会派生出 SSK，并保存在静态全局变量 tee_fs_ssk 中。SSK 作为 OPTEE 安全存储子系统的根密钥进行保存，用于派生其他密钥。

SSK 的派生规则是：

$$SSK = HMAC_{SHA256}(HUK, Chip\ ID \| \text{“static string”})$$

可以看到，其中 HUK（Hardware Unique Key）作为 key，Chip ID 和一个静态字符串作为内容，生成 sha256 的哈希值，便是 SSK。

HUK 从函数 tee_otp_get_hw_unique_key 中获取，E2000Q 中未实现 HUK 的直接读取，所以采用了一个全 0 的测试 HUK 作为替代。代码实现在 otp_stubs.c 中。HUK 一般是一个 128bit 大小的 key。

Chip ID 从函数 tee_otp_get_die_id 中获取，OPTEE 在测试过程中，E2000Q 未实现读取真实的 Die ID 的接口，采用 get_dummy_die_id 用一个假的 Die ID 替代，是一个 256bit 大小的数。即反复填写 BEEF 字符串 8 次得到的数。

静态字符串，OPTEE 默认是“ONLY_FOR_tee_fs_ssk”这个字符串。

在测试中，由于并没有获取芯片真正的 HUK 和 Chip ID，所以每次的 SSK 都是一样的。正常产品这里需要获取真正的 HUK 和 Chip ID 才能确保 SSK 的唯一性。

1.3.3 TSK

TSK 在 OPTEE 中的目的只是用于保护 FEK，所以 TSK 作为运算的中间 Key，并不会进行全局保存，在 FEK 运算生成后即会消失。也就是说，加密的 FEK 和加密文件一起保留下来，但 TSK 属于过程 key 被丢弃了，这样就切断了 SSK 和加密 FEK 的联系。后续 FEK 的解密必须使用对应的 TSK，必须再次生成同一个 SSK 才能正确解密，所以这样的规则就确保了加密的 FEK 和加密文件即便被拷贝出去了，也不能被解密。

TSK 的派生规则是：

$$TSK = HMAC_{SHA256}(SSK, TA_UUID)$$

可以看出 TSK 也是一个 256bit 大小的哈希值，SSK 作为 key，操作 TA 的 UUID 作为内容，进行

sha256 运算得到。

TSK 的生成代码在 tee_fs_fek_crypt 函数中，也就是生成 FEK 的过程中，作为局部变量存在。

1.3.4 FEK

FEK 作为安全存储中，文件加解密的密钥，是 Key manager 的重要组成部分。

代码实现在 tee_fs_key_manager.c 中。

下图是 RPMB 读写测试过程中一次 FEK 的获取 log：

```
rpmb_fs_open_internal:2315 GENERATE FEK key: 0xfc0cb9c0
rpmb_fs_open_internal:2317 00000000fc0cb9c0  59 95 e9 7d e5 06 a1 07  c2 0f c9 94 01 6c 1b 44
```

总共 16 个字节，128bit 大小。

实际的 FEK 是一个 128 位的伪随机数，系统的 crypto_rng_read 接口获取该随机数。

FEK 的关键在 tee_fs_fek_crypt 函数对这个伪随机数进行加密计算。

tee_fs_fek_crypt 函数先使用 SSK 和 UUID，通过 sha256 生成 TSK。然后用 TSK 作为 key 对输入的随机数进行 AES（ECB mode）加密计算，得到的密文作为 key 值输出。

上图 log 中，有红色显示的就是 RPMB 读写过程中经过加密后的 FEK 值。

1.4 Hashtree

安全存储中是以 Hashtree（哈希树）的方式进行文件管理的，每个已经加密的文件的文件名是一个递增的数字，而内容是密文形式存在。

具体 Hashtree 的组织原理并未深究，基本可以理解将一个文件放在一个二叉树的节点（Node）上，这个文件的内容就是实际的 Block 数据，而节点的信息就是文件对应的 Meta 数据。

节点 Meta 数据的生成流程图请参考 https://optee.readthedocs.io 文档中的 Hash Tree 小节的第一张图。

绿色是文件实际的 Meta 数据（明文），Encrypted FEK 就是 tee_fs_fek_crypt 函数中生成的密文值，Meta IV 是一个 128bit 的随机数，这里作为 AES（GCM mode）运算的初始向量值。

以 FEK 为 key，Meta IV 作为 iv 向量值，Encrypted FEK 作为 AAD（附加验证数据，可以作为生成校验码的盐值，不参与明文加密运算），Meta data 作为明文，进行 AES（GCM mode）加密运算。

最后保存 Encrypted FEK、Meta IV、加密后的密文以及 Tag 值（消息认证码，MAC），通过 RPC 发送到 REE 侧处理。

Block 数据的生成流程图请参考 https://optee.readthedocs.io 文档中的 Hash Tree 小节的第二张图。

这个纯数据的加密过程则比较简单，就是通过解密节点信息的 Encrypted FEK，得到最开始的 FEK 值，然后用 FEK 对 Block Data 进行 AES（GCM mode）加密运算，运算完成后，FEK 不用保存。最后结果只需要保存初始向量值 Block IV、消息认证码 Tag 值和加密后的密文数据。

2 RPMB

2.1 硬件属性

RPMB Partition 是 eMMC 中的一个具有安全特性的分区。用户在读写 eMMC 的 RPMB 分区时，会有校验等安全操作，以确保数据是 RPMB 内部数据，而不是攻击者伪造的数据。

可以看出 RPMB 作为 eMMC 的一个特殊分区，在两个启动分区之后。一般大小是 4MB，E2000Q 参考板实测 log 如下：

```
0 tee_rpmb_init:1113 RPMB: Syncing device information
0 tee_rpmb_init:1121 RPMB: RPMB size is 32*128 KB
0 tee_rpmb_init:1122 RPMB: Reliable Write Sector Count is 1
0 tee_rpmb_init:1149 RPMB INIT: Deriving key
```

在 eMMC 硬件设备中，和 RPMB 分区有关的硬件实体，主要有两个：一个是存放在 eMMC OTP 区域的 RPMB key，另一个是 RPMB 的写计数器。

关于 eMMC 的 OTP 区域，查询不到对应地址，因为 eMMC 也相当于一个小的嵌入式系统，个人估计 OTP 作为一个特殊设备存在于该嵌入式系统中，类似飞腾芯片内部的 eFuse。

eMMC 内部的地址划分，生产厂家并未公开，但和外界是通过 eMMC 标准的协议交互。这样，可以通过标准 eMMC 命令（set key）完成对 OTP 的一次烧写。注意，一旦完成烧写，就无法更改，即便 eMMC 中的厂家信息都是可以修改的，OTP 中的 Key 信息是绝对无法更改的。

如果 RPMB key 的生成和 CPU 的 HUK 或芯片 ID 进行了绑定，那么意味着 eMMC 的 RPMB 区域也和 CPU 进行了绑定，更换 CPU 将无法使用 RPMB 分区。

同样的，RPMB 的写计数器值也无法从公开的寄存器中直接读取，必须通过标准的 RPMB 读取流程完成读取（写计数器读命令）。写计数器的自加过程由 eMMC 硬件（也可能是内部嵌入式系统的软件，当检测到一次正确的写入完成后，对某 32bit 位宽的内部计数器进行+1 操作）完成。

2.2 读写流程

2.2.1 概述

RPMB 的读写驱动，在本文档描述的场景中，是由 Linux 内核的 eMMC 驱动完成，OPTEE 在 Linux 的后台程序 tee-supplicant 响应来自 TEE 侧的请求后，采用标准的 POSIX 接口读写方式，读写 RPMB

的设备文件/dev/mmcblk%urpmb，其中%u 是 eMMC 的 dev id，在 E2000Q 参考板中，dev id 是 0，即 eMMC 挂着 SD0 接口下。

虽然 RPMB 最底层的读写流程是由 Linux 对应的驱动完成，并不是本文档涉及的范围，但是研究这个流程有助于理解 RPMB 的安全性。

2.2.2 密钥

RPMB key 是存放在 eMMC 的 OTP 区域的一个密钥，该密钥的副本在 TEE 侧是可以明文保存的。OPTEE 中有写入程序可以将 HUK 派生出来的 RPMB key 写入到对应的 OTP 中，但注意，这是一次写入，不可更改的。作为产品，不建议用以上的方法写入这个 key，需要通过产线工具，规划好派生规则，然后统一写入。

RPMB 分区的数据读写都需要通过 RPMB key 做签名和验签。

2.2.3 读流程

RPMB 读流程：

1) Host 向 eMMC 发起读 RPMB 的请求（读取命令和目标数据的信息），同时生成一个 128bit 的随机数发送给 eMMC。

2) eMMC 将请求的目标数据从 RPMB 中读出，并使用 RPMB key（Secure Key）通过 HMAC SHA256 算法，对读取到的数据（Data）和接收到的随机数（Random Bytes）拼接到一起后运算得到 MAC 值。然后，eMMC 将读取到的数据、接收到的随机数和计算得到的 MAC 值一并发送给 Host。

3) Host 接收到 RPMB 的数据、随机数和 MAC 值后，首先比较随机数是否与自己发送的一致，如果一致，再用同样的 Secure Key 通过 sha256（HMAC）算法对数据和随机数计算 MAC，如果结果与 eMMC 发送的 MAC 是一致的，那么就可以确定该数据是从 RPMB 中读取到的正确数据，而不是攻击者伪造的数据。

2.2.4 写流程

RPMB 写流程：

1) 第一步是重复一个读流程，将 eMMC 中的写计数器值读出来。

2) 然后，Host 再将需要写入的数据和上一步读到的计数器值拼接到一起并使用 RPMB key 计算 MAC 值，再将数据（Data）、写计数器值（Write Cnt）以及 MAC 值一起发给 eMMC。

3) eMMC 接收到数据后，先对比计数器值是否与当前的值相同，如果相同那么再计算和对比 MAC 值，如果相同则将数据写入到 RPMB 分区中，否则不写入数据。

2.3 重放攻击

所谓重放攻击，简单理解就是：曾经的正确的请求数据被攻击者截获后，攻击者使用该请求数据发给目标，目标依然认为请求数据是合法的，通过这种攻击方式，攻击者建立和目标的合法连接，并获得目标提供的数据，甚至是管理权限。

RPMB 能阻止重放攻击的原理是：

在 eMMC 内部，有一个 RPMB 写计数器。RPMB 每进行一次合法的写入操作时，计数器就会自动加 1。通过 RPMB key 和写计数器的使用，RPMB 可以实现数据读取和写入时的重放保护(Replay Protect)，即攻击者无法使用曾经的正确请求信息再次建立合法连接，因为写计数器的值已经发生了变化。

以下是 E2000Q 板在调试 RPMB 过程中，读取到的 RPMB 部分信息的 log，其中红色画圈部分就是读出来的 RPMB 写计数器的值（16 进制），该值增加到 0xFFFFFFFF 后，RPMB 将无法写入，只能读取。

```
D/TC:? 0 tee_rpmb_init:1185 93 a6 59 d3
D/TC:? 0 tee_rpmb_init:1186 RPMB wr_cnt       : 107
D/TC:? 0 tee_rpmb_init:1187 RPMB max_blk_idx  : 3fff
D/TC:? 0 tee_rpmb_init:1188 RPMB rel_wr_blkcnt: 1
D/TC:? 0 tee_rpmb_init:1189 RPMB dev_id       : 0
D/TC:? 0 tee_fs_crypt_block:240 Encrypt block #16381
D/TC:? 0 tee_rpmb_write_blk:1403 Write 1 block at inde
D/TC:? 0 tee_rpmb_init:1180 RPMB info:
D/TC:? 0 tee_rpmb_init:1181 RPMB cid:
D/TC:? 0 tee_rpmb_init:1182 13 1 4e 47
D/TC:? 0 tee_rpmb_init:1183 31 4d 31 35
D/TC:? 0 tee_rpmb_init:1184 4c 10 21 26
D/TC:? 0 tee_rpmb_init:1185 93 a6 59 d3
D/TC:? 0 tee_rpmb_init:1186 RPMB wr_cnt       : 108
D/TC:? 0 tee_rpmb_init:1187 RPMB max_blk_idx  : 3fff
D/TC:? 0 tee_rpmb_init:1188 RPMB rel_wr_blkcnt: 1
D/TC:? 0 tee_rpmb_init:1189 RPMB dev_id       : 0
D/TC:? 0 tee_rpmb_init:1180 RPMB info:
D/TC:? 0 tee_rpmb_init:1181 RPMB cid:
D/TC:? 0 tee_rpmb_init:1182 13 1 4e 47
D/TC:? 0 tee_rpmb_init:1183 31 4d 31 35
D/TC:? 0 tee_rpmb_init:1184 4c 10 21 26
D/TC:? 0 tee_rpmb_init:1185 93 a6 59 d3
D/TC:? 0 tee_rpmb_init:1186 RPMB wr_cnt       : 109
D/TC:? 0 tee_rpmb_init:1187 RPMB max_blk_idx  : 3fff
```

2.4 增强安全性

OPTEE 安全存储的框架解决了存放在 REE 侧的数据应该是密文的问题，但是，即便是密文，从硬件（硬盘或 Flash）上讲，对存放在本实体上的数据并没有鉴权的操作，即所有人都可以读写。那这样，攻击者将密文数据，甚至是硬件实体移动到自己的硬件平台上，如果 SSK 并没有和原硬件平台进行绑定，或者是保护得不好的情况下，那么，攻击者在对安全存储的流程非常熟悉的基础上，通过某些攻击方式，理论上依然是有攻破的可能。

RPMB 分区的使用增强了 OPTEE 安全存储的框架，体现在对安全存储实体的安全性保护上。通过 RPMB 分区的硬件特性，确保密文数据存放位置不被非法改动。也就是说：

首先，攻击者无法读出密文数据来进行搬移，因为读取过程中需要 RPMB key 鉴权，攻击者的软件程序无法透过 ARM trustzone 的框架运行在安全世界，所以无法获取到 RPMB key。

其次，攻击者如果是将整个 eMMC 芯片采用物理方式移动到相似硬件平台下的话，由于 RPMB key 是存储在 eMMC 的 OTP 区域的，硬件保证并不能直接读取，所以在新的硬件平台下，攻击者即使得到 TEE 侧的运行权限，也无法获取或派生正确的 RPMB key，那这样 RPMB 的读写鉴权始终无法通过。

由此可见，RPMB key 是非常关键的一个环节，E2000Q 参考板的测试中，采用了一个固定的测试 key 值，而正常产品中，应该在产线工具中做好 RPMB key 的生成和烧写，规则由整机厂家确定好，比如可以使用 HUK 或芯片信息进行派生等。确保原板上的启动软件能够使用正确的规则，在 TEE 侧派生出正确的 RPMB key。

3 基于 xtest 的测试

3.1 前期配置

OPTEE 相关测试工具 xtest 支持安全存储的相关测试，其中 6000 系列的测试集就是和安全存储相关的，比如 6001 是测试创建文件，6002 测试打开文件等。相关源码可以参考开源项目 optee_test 中的 regression_6000.c 文件。通过该工具源码的研究，可以了解 TA 侧采用标准 GP 接口发起安全存储相关操作的方法，后面的实现在 libutee 库中。

基于 E2000Q 参考板和 OPTEE 3.12，采用 xtest 测试 RPMB，首先需要修改几个配置：

1) 在 OPTEE OS 配置中打开 RPMB 的设置，即改变安全存储默认 REE_FS 为 RPMB_FS。即在对应的 mk 文件中增加以下配置：

```
$(call force,CFG_RPMB_FS,y)
```

2) 在 OPTEE Client 配置文件 tee-supplicant/Makefile 中，修改 RPMB_EMU 配置为 0，这里是因为 tee-supplicant 默认使用自带的一个 RPMB 仿真模块，用内存进行 RPMB 操作的模拟仿真，配置为 0 后，关闭该模块，采用真正的 RPMB 操作。

3) 另外 OPTEE OS 中，对应的 mk 文件还有两个配置：

```
$(call force,CFG_RPMB_TESTKEY,y)
$(call force,CFG_RPMB_WRITE_KEY,n)
```

可以根据需要选 y 或 n，CFG_RPMB_TESTKEY 选 y 后，系统会使用 OPTEE 提供的一个固定的 RPMB key，这样不需要从 HUK 中去派生 key。CFG_RPMB_WRITE_KEY 选 y 后，OPTEE 会发出写 RPMB key 的命令给 REE 侧，如果 REE 侧是真实的 RPMB 硬件，那么要特别注意，只能写一次，而且写了之后，这个 eMMC 的 RPMB key 就永远是这次写入的这个了，无法再改变了，请注意 Key 一定要能正确得保存好，或能正确得派生出来，否则这个 eMMC 的 RPMB 就废了。

测试的时候，可以写入 OPTEE 提供的 RPMB key，OPTEE 官方源码已经公布的，固化在程序代码中，不存在遗失的问题，当然，这仅仅用于测试。

3.2 常见 log 和错误说明

3.2.1 测试 log

基于 E2000Q 参考板上进行一次完整的 xtest 6001 到 6010 测试。

以下截取 xtest 6001.1 测试子项（创建一个安全文件）log：

```
* regression_6001 Test TEE_CreatePersistentObject
o regression_6001.1 Storage id: 00000001
D/TC:3 0 core_mmu_entry_to_finer_grained:750 xlat tables used 3 / 5
D/TC:? 0 tee_ta_init_pseudo_ta_session:299 Lookup pseudo TA
b689f2a7-8adf-477a-9f99-32e90c0ad0a2
D/TC:? 0 ldelf_load_ldelf:91 ldelf load address 0x40006000
D/LD:   ldelf:134 Loading TA b689f2a7-8adf-477a-9f99-32e90c0ad0a2
F/TC:? 0 trace_syscall:151 syscall #3 (syscall_get_property)
F/TC:? 0 trace_syscall:151 syscall #5 (syscall_open_ta_session)
D/TC:? 0 ldelf_syscall_open_bin:140 Lookup user TA ELF
b689f2a7-8adf-477a-9f99-32e90c0ad0a2 (Secure Storage TA)
D/TC:? 0 tee_rpmb_init:1113 RPMB: Syncing device information
D/TC:? 0 tee_rpmb_init:1121 RPMB: RPMB size is 32*128 KB
D/TC:? 0 tee_rpmb_init:1122 RPMB: Reliable Write Sector Count is 1
D/TC:? 0 tee_rpmb_init:1149 RPMB INIT: Deriving key
D/TC:? 0 tee_rpmb_key_gen:301 RPMB: Using test key
D/TC:? 0 tee_rpmb_init:1164 RPMB INIT: Verifying Key
D/TC:? 0 tee_rpmb_read:1249 Read 1 block at index 0
D/TC:? 0 tee_rpmb_write_blk:1392 Write 1 block at index 0
D/TC:? 0 tee_rpmb_read:1249 Read 1 block at index 0
D/TC:? 0 tee_rpmb_read:1249 Read 8 blocks at index 2
F/TC:? 0 dump_fat:1896 flags 0x1, size 32, address 0x3ffe00, filename '/dirfile.db.hash'
F/TC:? 0 dump_fat:1896 flags 0, size 0, address 0, filename ''
F/TC:? 0 dump_fat:1896 flags 0, size 0, address 0, filename ''
F/TC:? 0 dump_fat:1896 flags 0, size 0, address 0, filename ''
F/TC:? 0 dump_fat:1896 flags 0x2, size 0, address 0, filename ''
D/TC:? 0 read_fat:2136 fat_address 0
D/TC:? 0 tee_rpmb_read:1249 Read 8 blocks at index 2
D/TC:? 0 dump_fh:1912 fh->filename=/dirfile.db.hash
D/TC:? 0 dump_fh:1913 fh->rpmb_fat_address=512
D/TC:? 0 dump_fh:1914 fh->fat_entry.start_address=4193792
D/TC:? 0 dump_fh:1915 fh->fat_entry.data_size=32
D/TC:? 0 read_fat:2136 fat_address 512
D/TC:? 0 tee_rpmb_read:1249 Read 8 blocks at index 2
D/TC:? 0 tee_rpmb_read:1249 Read 1 block at index 16382
D/TC:? 0 tee_fs_crypt_block:240 Decrypt block #16382
D/TC:? 0 ldelf_syscall_open_bin:144 res=0xffff0008
D/TC:? 0 ldelf_syscall_open_bin:140 Lookup user TA ELF
b689f2a7-8adf-477a-9f99-32e90c0ad0a2 (REE)
D/TC:? 0 ldelf_syscall_open_bin:144 res=0
F/TC:? 0 trace_syscall:151 syscall #7 (syscall_invoke_ta_command)
```

```
F/TC:? 0 trace_syscall:151 syscall #11 (syscall_mask_cancellation)
F/TC:? 0 trace_syscall:151 syscall #7 (syscall_invoke_ta_command)
F/TC:? 0 trace_syscall:151 syscall #3 (syscall_get_property)
F/TC:? 0 trace_syscall:151 syscall #8 (syscall_check_access_rights)
F/TC:? 0 trace_syscall:151 syscall #8 (syscall_check_access_rights)
D/TC:? 0 read_fat:2136 fat_address 0
D/TC:? 0 tee_rpmb_read:1249 Read 8 blocks at index 2
D/TC:? 0 dump_fh:1912 fh->filename=/dirfile.db.hash
D/TC:? 0 dump_fh:1913 fh->rpmb_fat_address=512
D/TC:? 0 dump_fh:1914 fh->fat_entry.start_address=4193792
D/TC:? 0 dump_fh:1915 fh->fat_entry.data_size=32
D/TC:? 0 read_fat:2136 fat_address 512
D/TC:? 0 tee_rpmb_read:1249 Read 8 blocks at index 2
D/TC:? 0 tee_rpmb_read:1249 Read 1 block at index 16382
D/TC:? 0 tee_fs_crypt_block:240 Decrypt block #16382
F/TC:? 0 trace_syscall:151 syscall #6 (syscall_close_ta_session)
F/TC:? 0 trace_syscall:151 syscall #3 (syscall_get_property)
D/LD:   ldelf:168 ELF (b689f2a7-8adf-477a-9f99-32e90c0ad0a2) at 0x40098000
F/TC:? 0 trace_syscall:151 syscall #8 (syscall_check_access_rights)
F/TC:? 0 trace_syscall:151 syscall #42 (syscall_storage_obj_create)
D/TC:? 0 read_fat:2136 fat_address 0
D/TC:? 0 tee_rpmb_read:1249 Read 8 blocks at index 2
D/TC:? 0 dump_fh:1912 fh->filename=/dirfile.db.hash
D/TC:? 0 dump_fh:1913 fh->rpmb_fat_address=512
D/TC:? 0 dump_fh:1914 fh->fat_entry.start_address=4193792
D/TC:? 0 dump_fh:1915 fh->fat_entry.data_size=32
D/TC:? 0 read_fat:2136 fat_address 512
D/TC:? 0 tee_rpmb_read:1249 Read 8 blocks at index 2
D/TC:? 0 tee_rpmb_read:1249 Read 1 block at index 16382
D/TC:? 0 tee_fs_crypt_block:240 Decrypt block #16382
D/TC:? 0 dump_fh:1912 fh->filename=/dirfile.db.hash
D/TC:? 0 dump_fh:1913 fh->rpmb_fat_address=512
D/TC:? 0 dump_fh:1914 fh->fat_entry.start_address=4193792
D/TC:? 0 dump_fh:1915 fh->fat_entry.data_size=32
D/TC:? 0 read_fat:2136 fat_address 512
D/TC:? 0 tee_rpmb_read:1249 Read 8 blocks at index 2
D/TC:? 0 rpmb_fs_write_primitive:2501 Updating data in-place
D/TC:? 0 tee_rpmb_read:1249 Read 1 block at index 16382
D/TC:? 0 tee_fs_crypt_block:240 Decrypt block #16382
D/TC:? 0 tee_rpmb_write_blk:1392 Write 1 block at index 16382
D/TC:? 0 tee_fs_crypt_block:240 Encrypt block #16382
F/TC:? 0 trace_syscall:151 syscall #43 (syscall_storage_obj_del)
```

```
D/TC:? 0 dump_fh:1912 fh->filename=/dirfile.db.hash
D/TC:? 0 dump_fh:1913 fh->rpmb_fat_address=512
D/TC:? 0 dump_fh:1914 fh->fat_entry.start_address=4193792
D/TC:? 0 dump_fh:1915 fh->fat_entry.data_size=32
D/TC:? 0 read_fat:2136 fat_address 512
D/TC:? 0 tee_rpmb_read:1249 Read 8 blocks at index 2
D/TC:? 0 rpmb_fs_write_primitive:2501 Updating data in-place
D/TC:? 0 tee_rpmb_read:1249 Read 1 block at index 16382
D/TC:? 0 tee_fs_crypt_block:240 Decrypt block #16382
D/TC:? 0 tee_rpmb_write_blk:1392 Write 1 block at index 16382
D/TC:? 0 tee_fs_crypt_block:240 Encrypt block #16382
D/TC:? 0 tee_ta_close_session:514 csess 0xfc0cca90 id 1
D/TC:? 0 tee_ta_close_session:533 Destroy session
D/TC:? 0 destroy_context:310 Destroy TA ctx (0xfc0cca30)
  regression_6001.1 OK
```

3.2.2 OTP 没有 key 的错误

如果 eMMC 的 RPMB 是第一次使用，OTP 中没有 RPMB key，运行任何一个 xtest 6000 的测试子集，都会报“Auth key not yet written”的错误。如下图所示：

```
E/TC:? 0 tee_rpmb_verify_key_sync_counter:1012 Verify key returning 0xffff0008
D/TC:? 0 tee_rpmb_init:1172 RPMB INIT: Auth key not yet written
D/TC:? 0 tee_rpmb_write_and_verify_key:1074 RPMB INIT: Writing Key value:
D/TC:? 0 tee_rpmb_write_and_verify_key:1075 00000000fc0cbf70  d3 eb 3e c3 6e 33 4c
```

3.2.3 RPMB key 验证失败的错误

如果 eMMC 的 RPMB key 已经写入，但 OPTEE 在启动时派生出错误的 RPMB key 值，那么 xtest 6000 的任何测试子集在运行时会有“Verify key Failed”的错误提示。如下图所示：

```
E/TC:? 0 tee_rpmb_verify_key_sync_counter:1012 Verify key returning 0xffff000f
E/TC:? 0 tee_rpmb_init:1175 Verify key failed!
E/TC:? 0 tee_rpmb_init:1176 Make sure key here matches device key
D/TC:? 0 ldelf_syscall_open_bin:144 res=0xffff000f
```

3.2.4 RPMB_EMU 忘记关闭的错误

如果 tee-supplicant 中的 RPMB 模拟模块是打开的，即操作对象并不是真正的 RPMB 分区，可以在 Log 中看到 RPMB size 是 256KB，这不是一个正常的 RPMB 分区大小（正常是 4M 或更大），这是模拟模块随意提供的一个查询反馈值。如下图所示：

```
D/TC:? 0 tee_rpmb_init:1113 RPMB: Syncing device information
D/TC:? 0 tee_rpmb_init:1121 RPMB: RPMB size is 2*128 KB
D/TC:? 0 tee_rpmb_init:1122 RPMB: Reliable Write Sector Count is 1
```

A 术语和缩略语

名称	全称	解释
OPTEE	Open Portable TEE	一个开源的、可信执行环境运行软件实现的项目，包括操作系统、运行在非安全执行环境中的交互驱动、支持程序和库。
TEE	Trusted Execution Environment	可信的执行环境，在 ARM 的安全架构中，属于安全世界的执行环境。
REE	Rich Execution Environment	丰富的执行环境，在 ARM 的安全架构中，属于非安全世界的执行环境。
TA	Trusted Application	运行在安全世界的应用程序。
CA	Client Application	运行在非安全世界的应用程序，负责和安全世界的应用程序进行交互。
RPMB	Replay Protected Memory Block	重放保护内存块，这是 eMMC 设备中的一个具有安全特性的分区
eMMC	Embedded Multi Media Card	嵌入式多媒体卡，是一种应用嵌入式领域的内嵌式存储器标准规格，由 MMC 行业协会订立。
OTP	One Time Programmable memory	一次性写入的存储空间，特性是只能写入一次，无法更改，可以读取。

B 更新记录

更新日期	版本号	编制人	说明
2023-03-17	1.0	邓强	首次发布。
2023-05-06	1.1	邓强	采用新模板，优化字体。
2023-05-12	1.2	邓强	修改特别提示，更新部分样式，增加缩略语说明。